# Universidad Nacional Autónoma de México
## Facultad de Ciencias

Computación Distribuida

Práctica 01

## Introducción a la Computación Distribuida

**Profesores:**

Fernando Michel Tavera
Luis Mario Escobar Rosales
David Ortega Medina

Fecha de entrega
Lunes, 02 de Marzo del 2026.

# Lineamientos

Los alumnxs deberán hacer y entregar las pŕacticas siguiendo todos los puntos mencionados,si alguno llegase a ser incumplido la penalización puede variar desde puntos menos a su calificación hasta la nulificación total de éstas.

1. Las prácticas deben ser realizadas individualmente.
2. Las prácticas se deberán entregar a través de classroom, en un archivo .zip.
3. El nombre del archivo .zip, debe empezar con el número de la práctica, seguido del nombre del integrante.

   Practica1_FernandoMichel.zip

4. Se deberán usar únicamente las bibliotecas permitidas para dicha práctica.Queda prohibido el uso de cualquier biblioteca externa en el código.
5. Se deberá realizar un reporte en un archivo pdf que debe llevar lo siguiente :
   - Una descripción general de como se desarrolló la práctica.
   - Se debe hacer un análisis detallado en como se implementaron los algoritmos solicitados.Mencionando todas las consideraciones, etc.
   - Cualquier otro comentario o aclaración que consideren pertinente.

   Este pdf debe ir al mismo nivel que la carpeta principal de sus programas.
6. Se debe agregar un README.txt que contenga :
   - Número de la práctica
   - Nombre y número de cuenta

   Debe ir al mismo nivel(en la jerarquía de carpetas que el reporte).
7. Si ni el README ni el reporte llevan el nombre, habrá una penalización de la calificación con un punto menos.
8. Queda estrictamente prohibido el uso de cualquier código generado por Inteligencia Artificial, así como código copiado de Internet y copias entre equipos.De haber sospecha por alguna de estas situaciones, los involucrados deberán tener una entrevista con el ayudante de laboratorio.En esta entrevistas se les cuestionara aspectos de su implementaciones, algoritmos y código en general. En caso de incumplirse esta norma, la calificación de dicha práctica será automáticamente 0.
9. Si se tiene alguna complicación con la fecha y horario de entrega de la práctica , se debe avisar al ayudante para buscar una solución.
10. Si se entrega código que no compile y/o muera nada más ejecutarlo, la calificación será 0.

# Computación distribuida - Algoritmos básicos

**Nota :** En su implementación, no deberán importar bibliotecas o usar otros archivos a menos que la práctica o el profesor lo indique.Para esta práctica bastará con usar Simpy , time y pytest.

En esta práctica implementaremos algoritmos y estructuras básicas en la computación distribuida.

1. Implementar las interfaces de Nodo y Canal en las siguientes clases

   - CanalBroadcast
   - NodoVecinos (ALgoritmo 1 )
   - NodoTopologia (Algoritmo 2)
   - NodoBroadCast (Algoritmo 3)

   Además completa la función de cada clase según el tipo de Nodo siguiendo sus respectivos algoritmos.

2. Clases y variables

   - Prueba para conocer a los vecinos de los vecinos de un nodo identifiers (n.identifiers): Dicha variable debe corresponder con una lista donde cada nodo almacena a los otros nodos.La prueba arroja un error mostrando el nodo que tiene mal dicho valor de la variable en cuestión.
   - Prueba para el algoritmo Broadcast mensaje (nodo.mensaje): variable que hace referencia al mensaje que fue enviado por el nodo $p_s$

3. Uso Para el uso de las pruebas sigue los siguientes pasos:

   - Localiza en la misma carpeta que tus códigos fuentes el archivo test.py
   - Ejecuta el siguiente comando:

     myUser:$ pytest -q test.py

# Algoritmos

**Algorithm 1** Código vecinos para proceso $p_i$

$id_i = i$
$neighbors_i = \{\text{Conjunto de vecinos de } p_i\}$
$identifiers_i = \{\emptyset\}$

**main**()
**for all** $j \in neighbors_i$ **do**
    **send** MYNAME($neighbors_i$) to $j$
**end for**

**when** MYNAME($identifiers_j$) is received from neighbor $p_j$:
$identifiers_i = identifiers_i \cup identifiers_j$

**Algorithm 2** Código vecinos topología para $p_i$

$vecinos_i, proc_conocidos_i = \{i\}$
$canales_conocidos_i = \{< i, j >| \ j \in vecinos_i\}$

**start**()

**for all** $j \in vecinos_i$ **do**
    **send** POSITION($i, vecinos_i$) to $j$
**end for**

**when** POSITION($k, vecinos$) is received from neighbor $p_j$
**if** $k \notin proc_conocidos_i$ **then**
    $proc_conocidos_i = proc_conocidos_i \cup \{k\}$
    $canales_conocidos_i = canales_conocidos_i \cup \{< k, l >| \ l \in vecinos\}$
    **for all** $l \in vecinos_i \setminus \{j\}$ **do**
        **send** POSITION($k, vecinos$) to $l$
    **end for**
    **if** $\forall < l, m > \in canales_conocidos_i : \{l, m\} \subset proc_conocidos_i$ **then**
        $p_i$ conoce la gráfica de comunicación; **return**
    **end if**
**end if**

**Algorithm 3** Broadcast ingenuo para el proceso $p_i$

```
neighbors_i = {conjunto de vecinos}
seen_message = false
if p_i = p_s then
    seen_message = true
    send M to all neighbors
end if

when M is received from neighbor p_j:
if seen_message = false then
    seen_message = true
    send M to all neighbors
end if
```